DATA ANALYST - COMMODITY PRICE FORECASTING AI AGENT

Category: Commodity Analytics | Machine Learning | Time Series Forecasting | Python/R/DAX

Timeline: 15 Days | AI-Accelerated Innovation & Research Project

Target: Undergraduate / MBA / CA (Inter)

TASK:

Build a comprehensive silver commodity price forecasting system using LSTM neural networks, ARIMA/SARIMA models, Facebook Prophet, XGBoost gradient boosting, and advanced ensemble methods. Work with 25+ years of historical silver market data including daily OHLCV prices, macroeconomic indicators, futures term structure, supply-demand fundamentals, and market sentiment data. Develop interactive Power BI dashboards with advanced DAX expressions for real-time portfolio monitoring, risk analytics, and trade signal generation. Design a gamified Commodity Trading Arena simulation platform that trains data analysts through realistic market scenarios including crisis events, flash crashes, and manipulation attempts.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

PART F: SUBMISSION PROTOCOL

Important Note:

This Project is designed, developed, and administered solely by Zetheta Algorithms Private Limited ("Zetheta") for the purpose of assessing candidates, students, or event participants on behalf of engaging employers, institutes, or event organisers. Completion of this Project does not guarantee employment, academic outcomes, awards, or any other result - all such decisions remain at the sole discretion of the respective engaging party. Zetheta may use AI-assisted tools to evaluate submissions; such outputs are indicative in nature and may be inaccurate or incomplete. By participating, you consent to your performance data, scores, and submissions being shared with relevant engaging parties for assessment and decision-making purposes.

EXECUTIVE SUMMARY

Commodity Price Forecasting AI Agent is an enterprise-scale machine learning project where data analyst interns build a comprehensive silver price prediction system combining deep learning (LSTM neural networks), classical time series models (ARIMA/SARIMA), modern forecasting frameworks (Facebook Prophet), and gradient boosting methods (XGBoost). Working with 25+ years of real-world silver market data spanning daily price movements, macroeconomic indicators, futures term structure, supply-demand fundamentals, and market sentiment signals, participants develop an end-to-end forecasting pipeline that would typically require years of quantitative finance experience to master.

The project transforms raw commodity market data - including 6,783 daily OHLCV records, 312 monthly macroeconomic observations, 27,390 futures contract records, 26 years of supply-demand fundamentals, and 1,043 weekly sentiment readings - into actionable price forecasts and risk analytics through advanced Python/R implementations and Power BI dashboards with complex DAX expressions.

The project spans six comprehensive parts: Part A provides deep-dive training material on commodity market fundamentals, time series theory, machine learning architectures, and feature engineering techniques specific to precious metals; Part B presents the Commodity Trading Arena gamified simulation business concept; Part C applies the analytics framework to real-world silver market crises including the Hunt Brothers manipulation (1979–1980), the 2011 silver bubble, the COVID-19 crash, and the 2021 Reddit Silver Squeeze; Part D provides a detailed 15-day day-by-day development methodology; Part E guides interns to the best no-code tools and data sources; and Part F specifies the 1000-point assessment criteria.

The core challenge students must solve: How does a data analyst transform raw historical price data, macroeconomic signals, futures market structure, and sentiment indicators into reliable short-term and medium-term price forecasts that can inform trading decisions, hedging strategies, and portfolio risk management? The answer lies in mastering the complete ML pipeline - data preprocessing and stationarity testing, feature engineering with technical indicators and lagged variables, model selection and hyperparameter tuning, walk-forward validation with expanding windows, ensemble model construction, and performance evaluation using RMSE/MAE/MAPE/directional accuracy metrics - combined with deep domain knowledge of silver market dynamics, COMEX/LBMA market microstructure, industrial demand drivers, and monetary policy transmission mechanisms.

Students will also design Commodity Trading Arena - a browser-based gamified simulation platform that trains data analysts through realistic, time-pressured, scenario-driven challenges covering the full spectrum of commodity price analytics: from building basic moving average crossover signals to navigating flash crashes and market manipulation events under extreme time pressure.

PART A: TRAINING MATERIAL & LEARNING RESOURCES

SECTION A1: FOUNDATIONAL CONCEPTS

A1.1 Silver Market Fundamentals

What is Silver as a Commodity?

Silver occupies a unique dual role in global markets: it functions simultaneously as a precious metal (store of value, monetary asset, inflation hedge) and as an industrial metal (electronics, solar panels, medical applications, automotive). This duality makes silver price prediction fundamentally more complex than forecasting purely industrial commodities (like copper or aluminium) or purely monetary metals (like gold). The price of silver is influenced by a diverse set of factors spanning monetary policy, industrial production cycles, technological innovation, investment flows, and mine supply dynamics.

Key Market Characteristics:

- Market Size: The global silver market is approximately $30–40 billion annually in physical supply, with paper derivatives markets many multiples larger
- Primary Exchanges: COMEX (CME Group, New York), LBMA (London Bullion Market Association), MCX (Multi Commodity Exchange, India), TOCOM (Tokyo)
- Price Discovery: LBMA Silver Price (daily auction at 12:00 noon London time), COMEX front-month futures settlement
- Unit of Trading: Troy ounce (31.1035 grams); COMEX contract = 5,000 troy ounces; MCX Silver contract = 30 kg
- Price Quotation: USD per troy ounce (international), INR per kilogram (India)
- Historical Price Range (2000–2025): $4.07 (November 2001) to $49.82 (April 2011)
- Average Annual Volatility: 25–40%, significantly higher than gold (15–25%)

A1.2 Silver Supply Chain

Understanding the silver supply chain is critical for feature engineering in forecasting models. Supply-side shocks directly impact prices and create regime changes that pure technical models cannot capture.

Primary Supply Sources:

Supply Source	Annual Volume	Details
Primary Mine Production	~830 MOz (2024)	Mexico, Peru, China, Russia, Poland account for ~60%
Secondary (Scrap) Recovery	~150–180 MOz	Recycled from industrial waste, photographic, jewelry
Government Sales	Variable	Historically significant (China, India); now minimal
Producer Hedging	±20–40 MOz	Forward sales by miners; can add or reduce supply

Demand Categories:

Demand Segment	Annual Volume	Key Drivers
Industrial Fabrication	~550 MOz	Electronics, brazing alloys, catalysts, chemicals
Solar Photovoltaics	~200 MOz	Fastest-growing segment; each GW solar needs ~0.7 MOz
Jewelry & Silverware	~250 MOz	India is largest market; cultural and seasonal demand
Physical Investment	~200–300 MOz	Coins, bars, ETF holdings (SLV, PSLV, ZKB)
Photography	~25 MOz	Declining since digital revolution; minimal by 2025
EV & 5G Technology	~50 MOz	Emerging demand from electric vehicles, 5G infrastructure

A1.3 Price Drivers Framework

A comprehensive price forecasting model must incorporate multiple categories of price drivers. The following framework categorises the variables that interns must consider when building their feature sets:

Category	Key Variables	Relationship to Silver Price
Monetary Policy	Fed Funds Rate, ECB rates, real interest rates, QE/QT cycles	Inverse: lower rates = higher silver
Currency Markets	DXY (Dollar Index), USD/INR, EUR/USD	Inverse: weaker dollar = higher silver (USD-denominated)
Inflation Expectations	CPI, PCE, breakeven inflation rates, TIPS spreads	Positive: higher inflation = higher silver demand as hedge
Gold Price	LBMA Gold Price, Gold/Silver ratio	Strong positive correlation (0.7–0.9); ratio mean-reverts
Industrial Activity	Global PMI, industrial production indices, auto sales	Positive: stronger industrial activity = higher demand
Energy Markets	Crude oil, natural gas (mining energy costs)	Positive: higher energy = higher production costs
Investment Flows	ETF holdings (SLV, PSLV), CFTC COT positioning	Positive: net long speculative positioning = bullish
Supply Disruptions	Mine strikes, country risk, environmental regulations	Positive: supply disruptions = price spikes
Technology Demand	Solar installations, EV production, 5G rollout	Positive: secular growth in industrial demand
Geopolitical Risk	VIX, safe-haven flows, sanctions, trade wars	Positive: increased uncertainty = safe-haven demand

SECTION A2: TIME SERIES ANALYSIS FUNDAMENTALS

A2.1 Stationarity and Unit Root Testing

Time series forecasting models require the underlying data to be stationary - meaning the statistical properties (mean, variance, autocorrelation) do not change over time. Raw silver prices are non-stationary (they exhibit trends, changing volatility regimes, and structural breaks). The first critical step in any forecasting pipeline is to test for stationarity and apply appropriate transformations.

Augmented Dickey-Fuller (ADF) Test:

The ADF test is the standard unit root test. The null hypothesis is that the series has a unit root (is non-stationary). If the test statistic is more negative than the critical value, we reject the null and conclude stationarity.

Python Implementation:

```
from statsmodels.tsa.stattools import adfuller
import pandas as pd

data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv',
parse_dates=['Date'])
data.set_index('Date', inplace=True)

# Test raw prices (expected: non-stationary)
result = adfuller(data['Close'].dropna())
print(f'ADF Statistic: {result[0]:.4f}')
print(f'p-value: {result[1]:.4f}')
print('Critical Values:')
for key, value in result[4].items():
    print(f'  {key}: {value:.4f}')

# Test log returns (expected: stationary)
log_returns = data['Log_Returns'].dropna()
result_lr = adfuller(log_returns)
print(f'\nLog Returns ADF Statistic: {result_lr[0]:.4f}')
print(f'Log Returns p-value: {result_lr[1]:.6f}')
```

R Implementation:

```
library(tseries)
library(forecast)

data <- read.csv('silver_daily_ohlcv_2000_2025.csv')
```

```
prices <- ts(data$Close, frequency = 252)  # trading days per year

# ADF test on raw prices
adf.test(prices)

# ADF test on log returns
log_returns <- diff(log(data$Close))
adf.test(log_returns)

# KPSS test (complementary: null = stationary)
kpss.test(prices)
kpss.test(log_returns)
```

A2.2 Differencing and Transformation

When raw prices are non-stationary, we apply transformations to achieve stationarity. The most common approaches for commodity prices are:

Transformation	Formula	Use Case	Order d
First Difference	ΔP_t = P_t - P_{t-1}	Removes linear trend; simple returns	1
Log Returns	r_t = ln(P_t / P_{t-1})	Normalises variance; preferred for financial data	1
Percentage Returns	R_t = (P_t - P_{t-1}) / P_{t-1}	Intuitive interpretation; not additive over time	1
Seasonal Differencing	Δ_s P_t = P_t - P_{t-s}	Removes seasonal patterns (s = period)	1
Box-Cox Transform	y = (x^λ - 1) / λ	Stabilises variance; λ estimated from data	0–1

A2.3 Autocorrelation Analysis (ACF/PACF)

The Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) are diagnostic tools for identifying the order of AR and MA components in ARIMA models. For silver price log returns, the ACF typically shows:

- Weak but statistically significant autocorrelation at lags 1–5 (short-term momentum/mean-reversion)
- Significant autocorrelation in squared returns at lags 1–20 (volatility clustering - GARCH effects)
- The PACF helps distinguish AR(p) order: significant spike at lag k with cutoff suggests AR(k)

Python Implementation:

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# ACF and PACF of log returns
plot_acf(log_returns, lags=40, ax=axes[0, 0], title='ACF: Log
Returns')
plot_pacf(log_returns, lags=40, ax=axes[0, 1], title='PACF: Log
Returns')

# ACF and PACF of squared returns (volatility clustering)
plot_acf(log_returns**2, lags=40, ax=axes[1, 0], title='ACF:
Squared Returns')
plot_pacf(log_returns**2, lags=40, ax=axes[1, 1], title='PACF:
Squared Returns')

plt.tight_layout()
plt.savefig('acf_pacf_analysis.png', dpi=150)
plt.show()
```

SECTION A3: MACHINE LEARNING MODELS FOR PRICE FORECASTING

A3.1 ARIMA/SARIMA Models

ARIMA (AutoRegressive Integrated Moving Average) is the foundational time series model. It combines three components: AR(p) - autoregressive terms that capture the linear dependence on past values; I(d) - integration (differencing) to achieve stationarity; MA(q) - moving average terms that capture dependence on past forecast errors. For silver prices, typical orders found through ADF testing and ACF/PACF analysis are ARIMA(2,1,2) or ARIMA(3,1,1) for daily data, and ARIMA(1,1,1) for monthly data.

SARIMA extends ARIMA with seasonal components: SARIMA(p,d,q)(P,D,Q)[s] where (P,D,Q) are seasonal AR, differencing, and MA orders, and s is the seasonal period. For silver, weekly seasonality (s=5 for trading days) and annual patterns (s=252) may be relevant.

Python Implementation - Auto ARIMA with Silver Data:

```
import pandas as pd
import numpy as np
from statsmodels.tsa.arima.model import ARIMA
from pmdarima import auto_arima
from sklearn.metrics import mean_squared_error, mean_absolute_error
import warnings
warnings.filterwarnings('ignore')

# Load silver daily data
data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv',
parse_dates=['Date'])
data.set_index('Date', inplace=True)
prices = data['Close']

# Train-test split (80-20)
split_idx = int(len(prices) * 0.8)
train, test = prices[:split_idx], prices[split_idx:]

# Auto ARIMA to find optimal parameters
auto_model = auto_arima(
    train,
    start_p=0, start_q=0,
    max_p=5, max_q=5,
    d=1,  # first differencing (non-stationary prices)
    seasonal=False,
    stepwise=True,
```

```
    suppress_warnings=True,
    information_criterion='aic',
    trace=True
)
print(f'Best ARIMA order: {auto_model.order}')
print(f'AIC: {auto_model.aic():.2f}')

# Walk-forward validation
history = list(train)
predictions = []
for t in range(len(test)):
    model = ARIMA(history, order=auto_model.order)
    model_fit = model.fit()
    yhat = model_fit.forecast(steps=1)[0]
    predictions.append(yhat)
    history.append(test.iloc[t])
    if t % 100 == 0:
        print(f'Step {t}/{len(test)} completed')

# Performance metrics
rmse = np.sqrt(mean_squared_error(test, predictions))
mae = mean_absolute_error(test, predictions)
mape = np.mean(np.abs((test.values - predictions) / test.values)) *
100
print(f'RMSE: ${rmse:.2f}')
print(f'MAE: ${mae:.2f}')
print(f'MAPE: {mape:.2f}%')
```

R Implementation:

```
library(forecast)
library(tseries)

data <- read.csv('silver_daily_ohlcv_2000_2025.csv')
prices <- ts(data$Close, frequency = 252)

# Train-test split
n <- length(prices)
train <- window(prices, end = c(1, floor(n * 0.8)))
test <- window(prices, start = c(1, floor(n * 0.8) + 1))

# Auto ARIMA
fit <- auto.arima(train, stepwise = TRUE, approximation = FALSE)
summary(fit)

# Forecast
```

```
fc <- forecast(fit, h = length(test))
accuracy(fc, test)
```

A3.2 LSTM (Long Short-Term Memory) Neural Networks

LSTM networks are a class of Recurrent Neural Networks (RNNs) specifically designed to learn long-range temporal dependencies. Unlike vanilla RNNs that suffer from vanishing/exploding gradients, LSTMs use a gating mechanism (forget gate, input gate, output gate) to selectively retain or discard information across time steps. For commodity price forecasting, LSTMs can capture complex non-linear patterns, regime changes, and multi-scale temporal dependencies that linear models like ARIMA cannot.

LSTM Architecture for Silver Price Forecasting:

- Input Layer: Multivariate time series with lookback window of 60 trading days
- Features: Close price, Volume, Technical indicators (RSI, MACD, Bollinger Bands), Macro variables
- LSTM Layer 1: 128 units with return_sequences=True (passes full sequence to next layer)
- Dropout Layer: 20% dropout to prevent overfitting
- LSTM Layer 2: 64 units (outputs only final hidden state)
- Dropout Layer: 20% dropout
- Dense Layer: 32 units with ReLU activation
- Output Layer: 1 unit (predicted next-day closing price)

Python Implementation - Complete LSTM Pipeline:

```
import pandas as pd
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout,
BatchNormalization
from tensorflow.keras.callbacks import EarlyStopping,
ReduceLROnPlateau
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler, RobustScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import matplotlib.pyplot as plt
```

```
# ============ DATA LOADING & FEATURE ENGINEERING ============
data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv',
parse_dates=['Date'])
data.set_index('Date', inplace=True)

# Technical Indicators
data['SMA_20'] = data['Close'].rolling(window=20).mean()
data['SMA_50'] = data['Close'].rolling(window=50).mean()
data['SMA_200'] = data['Close'].rolling(window=200).mean()
data['EMA_12'] = data['Close'].ewm(span=12).mean()
data['EMA_26'] = data['Close'].ewm(span=26).mean()
data['MACD'] = data['EMA_12'] - data['EMA_26']
data['MACD_Signal'] = data['MACD'].ewm(span=9).mean()

# RSI (14-period)
delta = data['Close'].diff()
gain = delta.where(delta > 0, 0).rolling(14).mean()
loss = (-delta.where(delta < 0, 0)).rolling(14).mean()
rs = gain / loss
data['RSI_14'] = 100 - (100 / (1 + rs))

# Bollinger Bands
data['BB_Middle'] = data['Close'].rolling(20).mean()
data['BB_Std'] = data['Close'].rolling(20).std()
data['BB_Upper'] = data['BB_Middle'] + 2 * data['BB_Std']
data['BB_Lower'] = data['BB_Middle'] - 2 * data['BB_Std']
data['BB_Width'] = (data['BB_Upper'] - data['BB_Lower']) /
data['BB_Middle']

# Average True Range (ATR)
high_low = data['High'] - data['Low']
high_close = abs(data['High'] - data['Close'].shift())
low_close = abs(data['Low'] - data['Close'].shift())
data['ATR_14'] = pd.concat([high_low, high_close, low_close],
axis=1).max(axis=1).rolling(14).mean()

# Volume features
data['Volume_SMA_20'] = data['Volume'].rolling(20).mean()
data['Volume_Ratio'] = data['Volume'] / data['Volume_SMA_20']

# Lagged features
for lag in [1, 2, 3, 5, 10, 20]:
    data[f'Return_Lag_{lag}'] = data['Returns_Pct'].shift(lag)

# Drop NaN rows
data.dropna(inplace=True)
```

```
# ============ FEATURE SELECTION ============
features = ['Close', 'Volume', 'VWAP', 'SMA_20', 'SMA_50',
'SMA_200',
            'MACD', 'MACD_Signal', 'RSI_14', 'BB_Width', 'ATR_14',
            'Volume_Ratio', 'Return_Lag_1', 'Return_Lag_5',
'Return_Lag_20']

# ============ SCALING ============
scaler = MinMaxScaler(feature_range=(0, 1))
scaled_data = scaler.fit_transform(data[features])

# Target scaler (for inverse transform)
target_scaler = MinMaxScaler(feature_range=(0, 1))
target_scaler.fit_transform(data[['Close']])

# ============ SEQUENCE CREATION ============
def create_sequences(data, lookback=60, forecast_horizon=1):
    X, y = [], []
    for i in range(lookback, len(data) - forecast_horizon + 1):
        X.append(data[i - lookback:i, :])
        y.append(data[i + forecast_horizon - 1, 0])  # Close price
is column 0
    return np.array(X), np.array(y)

LOOKBACK = 60
X, y = create_sequences(scaled_data, LOOKBACK)

# Train-validation-test split (70-15-15)
train_size = int(len(X) * 0.70)
val_size = int(len(X) * 0.15)
X_train, y_train = X[:train_size], y[:train_size]
X_val, y_val = X[train_size:train_size+val_size],
y[train_size:train_size+val_size]
X_test, y_test = X[train_size+val_size:], y[train_size+val_size:]

# ============ MODEL ARCHITECTURE ============
model = Sequential([
    LSTM(128, return_sequences=True, input_shape=(LOOKBACK,
len(features))),
    Dropout(0.2),
    BatchNormalization(),
    LSTM(64, return_sequences=False),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(1)
])
```

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='mean_squared_error',
    metrics=['mae']
)
model.summary()

# ============ TRAINING ============
callbacks = [
    EarlyStopping(monitor='val_loss', patience=10,
restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5,
min_lr=1e-6)
]

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    batch_size=32,
    callbacks=callbacks,
    verbose=1
)

# ============ EVALUATION ============
predictions_scaled = model.predict(X_test)
predictions =
target_scaler.inverse_transform(predictions_scaled.reshape(-1, 1))
actuals = target_scaler.inverse_transform(y_test.reshape(-1, 1))

rmse = np.sqrt(mean_squared_error(actuals, predictions))
mae = mean_absolute_error(actuals, predictions)
mape = np.mean(np.abs((actuals - predictions) / actuals)) * 100
directional = np.mean(np.sign(np.diff(actuals.flatten())) ==
np.sign(np.diff(predictions.flatten()))) * 100

print(f'LSTM Model Performance:')
print(f'RMSE: ${rmse:.2f}')
print(f'MAE: ${mae:.2f}')
print(f'MAPE: {mape:.2f}%')
print(f'Directional Accuracy: {directional:.1f}%')
```

R Implementation - LSTM with Keras:

```
library(keras)
library(tensorflow)

data <- read.csv('silver_daily_ohlcv_2000_2025.csv')
prices <- data$Close

# Scale to [0, 1]
min_price <- min(prices); max_price <- max(prices)
scaled <- (prices - min_price) / (max_price - min_price)

# Create sequences
lookback <- 60
create_dataset <- function(data, lookback) {
  X <- array(0, dim = c(length(data) - lookback, lookback, 1))
  y <- numeric(length(data) - lookback)
  for (i in 1:(length(data) - lookback)) {
    X[i,,1] <- data[i:(i + lookback - 1)]
    y[i] <- data[i + lookback]
  }
  list(X = X, y = y)
}

dataset <- create_dataset(scaled, lookback)
train_size <- floor(nrow(dataset$X) * 0.8)

X_train <- dataset$X[1:train_size,,, drop = FALSE]
y_train <- dataset$y[1:train_size]
X_test <- dataset$X[(train_size+1):nrow(dataset$X),,, drop = FALSE]
y_test <- dataset$y[(train_size+1):length(dataset$y)]

# Build LSTM model
model <- keras_model_sequential() %>%
  layer_lstm(units = 128, return_sequences = TRUE, input_shape =
c(lookback, 1)) %>%
  layer_dropout(rate = 0.2) %>%
  layer_lstm(units = 64) %>%
  layer_dropout(rate = 0.2) %>%
  layer_dense(units = 32, activation = 'relu') %>%
  layer_dense(units = 1) %>%
  compile(optimizer = optimizer_adam(learning_rate = 0.001), loss =
'mse')

model %>% fit(X_train, y_train, epochs = 50, batch_size = 32,
              validation_split = 0.15, callbacks = list(
```

```
                callback_early_stopping(patience = 10,
restore_best_weights = TRUE)))
```

A3.3 XGBoost Gradient Boosting

XGBoost (eXtreme Gradient Boosting) is a powerful ensemble method that builds decision trees sequentially, with each tree correcting the errors of the previous ensemble. For commodity price forecasting, XGBoost excels at capturing non-linear relationships between features and prices, handling mixed data types, and providing built-in feature importance rankings. Its advantages over LSTM include faster training, better interpretability, and native handling of missing values.

Python Implementation - XGBoost with Walk-Forward Validation:

```
import xgboost as xgb
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import mean_squared_error
import numpy as np
import pandas as pd

# Load and prepare features (reuse feature engineering from A3.2)
data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv',
parse_dates=['Date'])
data.set_index('Date', inplace=True)

# Add all technical indicators (same as LSTM section)
# ... [feature engineering code from A3.2] ...

# Create lag features for supervised learning
target = 'Close'
data['Target'] = data[target].shift(-1)  # predict next day
data.dropna(inplace=True)

features = [col for col in data.columns if col != 'Target']
X = data[features]
y = data['Target']

# Walk-forward validation with expanding window
tscv = TimeSeriesSplit(n_splits=5)
scores = []

for train_idx, test_idx in tscv.split(X):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
```

```
        model = xgb.XGBRegressor(
            n_estimators=500,
            max_depth=6,
            learning_rate=0.01,
            subsample=0.8,
            colsample_bytree=0.8,
            reg_alpha=0.1,
            reg_lambda=1.0,
            early_stopping_rounds=20,
            eval_metric='rmse'
        )
        model.fit(X_train, y_train, eval_set=[(X_test, y_test)],
    verbose=False)

        preds = model.predict(X_test)
        rmse = np.sqrt(mean_squared_error(y_test, preds))
        scores.append(rmse)
        print(f'Fold RMSE: ${rmse:.2f}')

    print(f'Mean CV RMSE: ${np.mean(scores):.2f} (+/-
    ${np.std(scores):.2f})')

    # Feature importance
    importance = model.feature_importances_
    feat_imp = pd.DataFrame({'Feature': features, 'Importance':
    importance})
    feat_imp = feat_imp.sort_values('Importance',
    ascending=False).head(15)
    print('\nTop 15 Features:')
    print(feat_imp.to_string(index=False))
```

Excel Solution - Regression-Based Forecasting:

For interns who need to demonstrate concepts without Python/R, Excel provides regression-based forecasting using built-in functions:

Step	Excel Implementation
1. Data Input	Import silver_daily_ohlcv_2000_2025.csv into a worksheet; sort by date ascending
2. Moving Averages	=AVERAGE(D2:D21) for SMA(20); use exponential smoothing via Data Analysis Toolpak

3. Linear Forecast	=FORECAST.LINEAR(target_date, known_y_range, known_x_range) for trend projection
4. ETS Forecast	=FORECAST.ETS(target_date, values, timeline, [seasonality], [data_completion])
5. Confidence Intervals	=FORECAST.ETS.CONFINT(target, values, timeline, confidence_level)
6. RSI Calculation	IF(L2>0, L2, 0) for gains; IF(L2<0, -L2, 0) for losses; RSI = 100 - 100/(1+RS)
7. Visualization	Insert Combo Chart: Actual prices + SMA(20) + SMA(50) + Forecast band

A3.4 Facebook Prophet

Prophet is an additive regression model developed by Meta for time series forecasting. It decomposes the time series into trend, seasonality (weekly, yearly), and holiday effects. For silver price forecasting, Prophet is particularly useful for capturing annual seasonal patterns (e.g., Indian wedding season demand, Chinese New Year), weekly trading patterns, and trend changes (changepoints) that correspond to monetary policy shifts or market regime changes.

Python Implementation:

```
from prophet import Prophet
import pandas as pd
import matplotlib.pyplot as plt

data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv')
df = data[['Date', 'Close']].rename(columns={'Date': 'ds', 'Close':
'y'})

# Add regressors from macro data
macro = pd.read_csv('silver_macroeconomic_monthly.csv')
# Merge monthly macro with daily prices (forward fill)
macro['ds'] = pd.to_datetime(macro['Date'])
df['ds'] = pd.to_datetime(df['ds'])
df = df.merge(macro[['ds', 'Gold_Price_USD', 'DXY_Index',
'VIX_Index']],
              on='ds', how='left').ffill()

# Train-test split
train = df[df['ds'] < '2023-01-01']
test = df[df['ds'] >= '2023-01-01']
```

```
# Initialize Prophet with custom settings for commodity prices
model = Prophet(
    changepoint_prior_scale=0.15,  # more flexible trend
    seasonality_prior_scale=10.0,
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False,
    changepoint_range=0.9,
    interval_width=0.95
)

# Add external regressors
model.add_regressor('Gold_Price_USD')
model.add_regressor('DXY_Index')
model.add_regressor('VIX_Index')

# Fit and predict
model.fit(train)
future = model.make_future_dataframe(periods=len(test))
# Add regressor values for future periods
future = future.merge(df[['ds', 'Gold_Price_USD', 'DXY_Index',
'VIX_Index']],
                      on='ds', how='left').ffill()
forecast = model.predict(future)

# Plot components
model.plot_components(forecast)
plt.savefig('prophet_components.png', dpi=150)
```

A3.5 Ensemble Model Construction

No single model consistently outperforms all others across all market conditions. Ensemble methods combine predictions from multiple models to achieve more robust and accurate forecasts. For silver price forecasting, we implement three ensemble strategies:

Ensemble Method	Description	Advantage	When to Use
Simple Average	Equal weight to all models	Baseline; reduces variance	Best when models are similarly accurate
Inverse-Error Weighted	Weight = 1/RMSE of validation period	Better models get higher weight	Default choice for production
Stacking (Meta-Learner)	Train Ridge/Lasso on model predictions	Learns optimal combination	Best when models capture different patterns
Dynamic Switching	Use best model per market regime	Adapts to conditions	Requires regime classification model

Python Implementation - Weighted Ensemble:

```
import numpy as np

# Assume predictions from each model on test set
# arima_preds, lstm_preds, xgb_preds, prophet_preds = ...

# Validation RMSE for each model
val_rmse = {
    'arima': 1.85,
    'lstm': 1.42,
    'xgboost': 1.35,
    'prophet': 1.95
}

# Inverse-error weights
inv_errors = {k: 1/v for k, v in val_rmse.items()}
total = sum(inv_errors.values())
weights = {k: v/total for k, v in inv_errors.items()}
print('Ensemble Weights:', {k: f'{v:.3f}' for k, v in
weights.items()})
```

```
# Weighted ensemble prediction
ensemble_pred = (weights['arima'] * arima_preds +
                 weights['lstm'] * lstm_preds +
                 weights['xgboost'] * xgb_preds +
                 weights['prophet'] * prophet_preds)

# Stacking meta-learner
from sklearn.linear_model import Ridge
meta_X = np.column_stack([arima_preds, lstm_preds, xgb_preds,
prophet_preds])
meta_model = Ridge(alpha=1.0)
meta_model.fit(meta_X[:split], y_test[:split])
stacked_pred = meta_model.predict(meta_X[split:])
```

SECTION A4: FEATURE ENGINEERING FOR SILVER PRICE FORECASTING

Feature engineering is the single most impactful step in building a successful forecasting model. The quality of features directly determines the upper bound of model performance. This section provides comprehensive coverage of feature categories specific to silver price prediction.

A4.1 Technical Indicator Features

Indicator	Full Name	Use in Silver Forecasting
SMA(20, 50, 200)	Simple Moving Average	Trend identification; golden/death cross signals
EMA(12, 26)	Exponential Moving Average	Faster trend response; MACD components
MACD & Signal Line	Moving Avg Convergence Divergence	Momentum; histogram for signal strength
RSI(14)	Relative Strength Index	Overbought (>70) / oversold (<30) conditions
Bollinger Bands(20,2)	Mean ± 2σ envelope	Volatility; squeeze/expansion signals
ATR(14)	Average True Range	Volatility proxy; position sizing input
Stochastic %K, %D	Price relative to range	Momentum oscillator; divergence signals
ADX(14)	Average Directional Index	Trend strength (>25 = trending)
OBV	On-Balance Volume	Volume-price trend confirmation
VWAP	Volume Weighted Avg Price	Institutional fair value; support/resistance
Ichimoku Cloud	Multi-component system	Trend, support, resistance, momentum in one indicator
Fibonacci Retracements	Key ratios (38.2%, 50%, 61.8%)	Support/resistance levels after moves

A4.2 Macroeconomic Features

The silver_macroeconomic_monthly.csv file contains 19 macroeconomic variables spanning January 2000 to December 2025 (312 monthly observations). These must be merged with daily price data using forward-fill methodology. Key engineered features include:

Feature	Calculation	Significance
Real Interest Rate	Fed_Funds_Rate - CPI_YoY	Most important single macro driver; negative real rates = bullish silver
Gold/Silver Ratio	Gold_Price / Silver_Price	Mean-reverts around 65–75; extreme readings signal relative value
DXY Momentum	ΔDXY / DXY (3-month change)	Dollar trend impacts all USD-priced commodities
Yield Curve Slope	US_10Y - Fed_Funds	Recession predictor; steepening = risk-on for commodities
M2 Growth Rate	ΔM2 / M2 (YoY)	Monetary expansion drives inflation expectations and silver demand
Oil/Silver Ratio	Crude_Oil / Silver_Price	Cross-commodity relative value; energy cost impact
VIX Level & Change	VIX, ΔVIX	Fear gauge; silver as safe haven during spikes
ETF Holdings Change	ΔETF_Holdings (MoM)	Investment demand proxy; SLV/PSLV flow indicator

A4.3 Futures Market Structure Features

The silver_futures_contracts.csv file provides term structure data for COMEX silver futures. Key features derived from this data capture market expectations and positioning:

- Basis (Futures - Spot): Positive basis (contango) indicates carrying costs and storage premium; negative basis (backwardation) signals immediate physical demand or supply squeeze
- Term Structure Slope: Calculated as (12-month futures - 1-month futures) / spot price; a steepening curve signals expected future price increases
- Roll Yield: The return from rolling futures contracts; negative in contango, positive in backwardation
- Open Interest Momentum: Changes in total open interest signal new money entering (rising OI) or positions being liquidated (falling OI)
- Volume/Open Interest Ratio: High ratios indicate active trading and potential price moves

A4.4 Sentiment & Positioning Features

The silver_sentiment_weekly.csv file provides CFTC Commitments of Traders positioning, news sentiment, social media activity, and options market data. These alternative data sources capture market psychology:

Feature	Calculation	Interpretation
CFTC Net Speculative Long	NonComm_Long - NonComm_Short	Extreme net longs = contrarian bearish; extreme net shorts = contrarian bullish
Commercial Hedger Net	Comm_Long - Comm_Short	Smart money indicator; commercials tend to be right at extremes
COT Index	(Current - Min) / (Max - Min) over 3 years	Normalised positioning; values >0.9 or <0.1 are extreme
News Sentiment Score	NLP-derived (-1 to +1)	Daily aggregated sentiment from commodity news feeds
Google Trends Index	0–100 search interest	Retail attention indicator; spikes during major moves
Put/Call Ratio	Put Volume / Call Volume	Options market sentiment; high PCR = bearish sentiment
Implied Volatility 30D	Options-implied expected move	Forward-looking uncertainty measure; vol surface shape
ETF Flow	Weekly inflow/outflow ($M)	Retail and institutional investment demand signal
Retail Bullion Premium	Physical price over spot (%)	Physical market tightness; premiums spike during shortages

SECTION A5: ADVANCED DAX ANALYTICS FOR COMMODITY DASHBOARDS

While Python and R are the primary tools for model building, Power BI with DAX is the standard enterprise tool for interactive dashboards and reporting. This section covers the advanced DAX measures required to build comprehensive silver price analytics dashboards.

A5.1 Core Price Analytics Measures

Weighted Average Price:

```
VWAP Measure =
DIVIDE(
    SUMX(
        SilverDaily,
        SilverDaily[VWAP] * SilverDaily[Volume]
    ),
    SUM(SilverDaily[Volume])
)
```

Rolling Volatility (20-Day Annualised):

```
Rolling Volatility 20D =
VAR CurrentDate = MAX(SilverDaily[Date])
VAR Last20Days =
    FILTER(
        ALL(SilverDaily),
        SilverDaily[Date] <= CurrentDate &&
        SilverDaily[Date] > CurrentDate - 28  -- approx 20 trading
days
    )
VAR StdDev =
    CALCULATE(
        STDEV.P(SilverDaily[Log_Returns]),
        Last20Days
    )
RETURN
    StdDev * SQRT(252)  -- annualise
```

Maximum Drawdown:

```
Max Drawdown =
VAR PriceTable =
    ADDCOLUMNS(
        SilverDaily,
        "RunningMax",
        CALCULATE(
            MAX(SilverDaily[Close]),
            FILTER(
                ALL(SilverDaily),
                SilverDaily[Date] <= EARLIER(SilverDaily[Date])
            )
        )
    )
VAR DrawdownTable =
    ADDCOLUMNS(
        PriceTable,
        "Drawdown",
        DIVIDE(
            [Close] - [RunningMax],
            [RunningMax]
        )
    )
RETURN
    MINX(DrawdownTable, [Drawdown])
```

Sharpe Ratio (Annualised):

```
Sharpe Ratio =
VAR AvgReturn = AVERAGE(SilverDaily[Returns_Pct]) / 100
VAR StdReturn = STDEV.P(SilverDaily[Returns_Pct]) / 100
VAR RiskFreeDaily = 0.05 / 252  -- assume 5% annual risk-free
VAR ExcessReturn = AvgReturn - RiskFreeDaily
RETURN
    DIVIDE(ExcessReturn, StdReturn) * SQRT(252)
```

RSI Measure (14-Period):

```
RSI 14 =
VAR CurrentDate = MAX(SilverDaily[Date])
VAR Last14Days =
    TOPN(
        14,
        FILTER(ALL(SilverDaily), SilverDaily[Date] <= CurrentDate),
        SilverDaily[Date], DESC
    )
VAR AvgGain =
    CALCULATE(
        AVERAGEX(
            FILTER(Last14Days, SilverDaily[Dollar_Change] > 0),
            SilverDaily[Dollar_Change]
        )
    )
VAR AvgLoss =
    CALCULATE(
        AVERAGEX(
            FILTER(Last14Days, SilverDaily[Dollar_Change] < 0),
            ABS(SilverDaily[Dollar_Change])
        )
    )
VAR RS = DIVIDE(AvgGain, AvgLoss)
RETURN
    100 - DIVIDE(100, 1 + RS)
```

A5.2 Cross-Dataset Analytics with DAX

Correlation with Gold (Rolling 60-Day):

```
Silver Gold Correlation 60D =
VAR CurrentDate = MAX(SilverDaily[Date])
VAR PeriodData =
    FILTER(
        ALL(MacroMonthly),
        MacroMonthly[Date] <= CurrentDate &&
        MacroMonthly[Date] > CurrentDate - 90
    )
VAR AvgSilver = AVERAGEX(PeriodData,
MacroMonthly[Silver_Price_USD])
VAR AvgGold = AVERAGEX(PeriodData, MacroMonthly[Gold_Price_USD])
VAR Covariance =
    AVERAGEX(
        PeriodData,
        (MacroMonthly[Silver_Price_USD] - AvgSilver) *
        (MacroMonthly[Gold_Price_USD] - AvgGold)
    )
VAR StdSilver = STDEV.P(MacroMonthly[Silver_Price_USD])
VAR StdGold = STDEV.P(MacroMonthly[Gold_Price_USD])
RETURN
    DIVIDE(Covariance, StdSilver * StdGold)
```

Supply-Demand Balance Impact:

```
Supply Demand Signal =
VAR CurrentYear = YEAR(MAX(SilverDaily[Date]))
VAR Balance =
    CALCULATE(
        SUM(SupplyDemand[Supply_Demand_Balance_MOz]),
        SupplyDemand[Year] = CurrentYear
    )
RETURN
    SWITCH(
        TRUE(),
        Balance > 50, "Surplus: Bearish Supply Signal",
        Balance > 0, "Slight Surplus: Neutral",
        Balance > -50, "Slight Deficit: Mildly Bullish",
        "Significant Deficit: Bullish Supply Signal"
    )
```

SECTION A6: COMPREHENSIVE DATA REQUIREMENTS

This project provides five comprehensive datasets totalling over 35,000 records. Understanding each dataset's structure, provenance, and analytical purpose is critical for building accurate forecasting models.

A6.1 silver_daily_ohlcv_2000_2025.csv (6,783 records)

The primary dataset: daily Open-High-Low-Close-Volume data for silver from January 2000 to December 2025. Each row represents one trading day (weekdays only, excluding US market holidays). This is the core training data for all ML models.

Field Name	Data Type	Description	Analytical Use
Date	DATE	Trading date (YYYY-MM-DD)	Index; time series ordering
Open	FLOAT	Opening price (USD/oz)	Gap analysis; overnight sentiment
High	FLOAT	Intraday high (USD/oz)	Resistance levels; ATR calculation
Low	FLOAT	Intraday low (USD/oz)	Support levels; ATR calculation
Close	FLOAT	Closing price (USD/oz)	Primary target variable for forecasting
Adj_Close	FLOAT	Adjusted close (USD/oz)	Accounts for splits/adjustments
Volume	INT	Daily trading volume (contracts)	Liquidity; volume-price confirmation
VWAP	FLOAT	Volume-weighted avg price	Institutional fair value reference
Returns_Pct	FLOAT	Daily percentage return	Model feature; performance metric
Log_Returns	FLOAT	Natural log return	Preferred for statistical models; additive
Dollar_Change	FLOAT	Absolute price change (USD)	Volatility in dollar terms
Intraday_Range	FLOAT	High minus Low (USD)	Daily volatility proxy
Intraday_Range_Pct	FLOAT	Range as % of Low	Normalised volatility measure

A6.2 silver_macroeconomic_monthly.csv (312 records)

Field	Type	Description	Use
Date	DATE	Month start (YYYY-MM-01)	Time alignment
Silver_Price_USD	FLOAT	Monthly avg silver price	Cross-reference with daily data
Gold_Price_USD	FLOAT	Monthly avg gold price	Precious metals correlation
Silver_Gold_Ratio	FLOAT	Gold / Silver price ratio	Relative value indicator
Fed_Funds_Rate	FLOAT	Federal Funds effective rate (%)	Monetary policy signal
US_10Y_Yield	FLOAT	10-Year Treasury yield (%)	Long-term rate expectations
Real_Interest_Rate	FLOAT	Fed rate minus CPI inflation (%)	Key silver price driver
US_CPI_Index	FLOAT	Consumer Price Index level	Inflation tracking
DXY_Index	FLOAT	US Dollar Index	Currency impact on silver
Crude_Oil_WTI	FLOAT	WTI crude oil (USD/barrel)	Energy/commodity complex
VIX_Index	FLOAT	CBOE Volatility Index	Market fear gauge
US_M2_Supply_Trillions	FLOAT	M2 money supply (USD trillions)	Monetary expansion
India_Repo_Rate	FLOAT	RBI repo rate (%)	India demand indicator
USD_INR	FLOAT	Exchange rate	India silver price conversion
Silver_Price_INR_Per_Kg	FLOAT	INR price per kilogram	MCX reference price
Global_PMI	FLOAT	Manufacturing PMI	Industrial demand proxy
US_Unemployment_Rate	FLOAT	Unemployment (%)	Economic health indicator
ETF_Silver_Holdings_MOz	FLOAT	Silver ETF holdings (million oz)	Investment demand

A6.3 silver_futures_contracts.csv (27,390 records)

COMEX silver futures data from 2005–2025, with five contract tenors (1, 2, 3, 6, 12 months to expiry) per trading day.

Field	Type	Description
Trade_Date	DATE	Trading date
Contract_Month	VARCHAR	Expiry month (YYYY-MM)
Months_To_Expiry	INT	Months until contract expires
Spot_Price	FLOAT	Spot silver price (USD/oz)
Futures_Price	FLOAT	Futures settlement price
Basis	FLOAT	Futures - Spot price
Basis_Pct	FLOAT	Basis as percentage of spot
Open_Interest	INT	Outstanding contracts
Volume	INT	Daily trading volume
Settlement_Price	FLOAT	Official settlement price
Implied_Carry_Rate_Annual	FLOAT	Annualised carry cost/yield (%)

A6.4 silver_supply_demand_annual.csv (26 records)

Annual silver supply and demand fundamentals from 2000–2025, sourced from the Silver Institute World Silver Survey framework.

Key fields include mine production by country (Mexico, Peru, China), demand by category (industrial, jewelry, investment, photography, silverware), emerging technology demand (solar PV, EV), exchange warehouse stocks, and LBMA vault holdings. The supply-demand balance is the critical fundamental signal.

A6.5 silver_sentiment_weekly.csv (1,043 records)

Weekly alternative data from 2006–2025 combining CFTC Commitments of Traders positioning, news sentiment, social media metrics, and options market indicators.

The CFTC data distinguishes between commercial hedgers (miners, fabricators, refiners) and non-commercial speculators (hedge funds, CTAs). Extreme positioning readings have historically preceded major silver price reversals. The COT Index normalises current positioning relative to its 3-year range.

SECTION A7: VOLATILITY MODELLING & RISK METRICS

A7.1 GARCH Models for Silver Volatility

Silver exhibits pronounced volatility clustering - periods of high volatility tend to be followed by more high volatility, and calm periods persist. This is captured by GARCH (Generalized Autoregressive Conditional Heteroscedasticity) models. Accurate volatility forecasting is essential for: (1) risk management and position sizing, (2) options pricing, (3) regime detection for switching between forecasting models, and (4) confidence interval estimation for price forecasts.

Python Implementation - GARCH(1,1) for Silver Volatility:

```
from arch import arch_model
import pandas as pd
import numpy as np

data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv',
parse_dates=['Date'])
data.set_index('Date', inplace=True)
returns = data['Returns_Pct'].dropna()

# Standard GARCH(1,1)
garch_model = arch_model(returns, vol='Garch', p=1, q=1, dist='t')
garch_fit = garch_model.fit(disp='off')
print(garch_fit.summary())

# EGARCH (asymmetric: captures leverage effect)
egarch_model = arch_model(returns, vol='EGARCH', p=1, q=1,
dist='t')
egarch_fit = egarch_model.fit(disp='off')

# GJR-GARCH (threshold GARCH for asymmetry)
gjr_model = arch_model(returns, vol='Garch', p=1, o=1, q=1,
dist='t')
gjr_fit = gjr_model.fit(disp='off')

# Forecast 30-day ahead volatility
forecast = garch_fit.forecast(horizon=30)
variance_30d = forecast.variance.iloc[-1].values
vol_30d_annualised = np.sqrt(variance_30d.mean()) * np.sqrt(252)
print(f'30-Day Annualised Volatility Forecast:
{vol_30d_annualised:.2f}%')

# Value at Risk (VaR)
from scipy.stats import norm, t as t_dist
```

```
mean_return = returns.mean()
vol_1d = garch_fit.conditional_volatility.iloc[-1]
var_95 = mean_return - 1.645 * vol_1d  # parametric VaR
var_99 = mean_return - 2.326 * vol_1d
print(f'1-Day 95% VaR: {var_95:.2f}%')
print(f'1-Day 99% VaR: {var_99:.2f}%')
```

A7.2 Risk Metrics for Commodity Portfolios

Interns must compute and understand the following risk metrics, all of which are standard in commodity trading desks and risk management frameworks:

Metric	Definition	Silver Context	Frequency
Value at Risk (VaR)	Maximum expected loss at confidence level	95% VaR = $1.50 means 5% chance of losing more than $1.50/oz	Daily/Weekly
Conditional VaR (CVaR)	Expected loss beyond VaR (tail risk)	Average of losses exceeding VaR; captures fat-tail risk	Daily/Weekly
Maximum Drawdown	Largest peak-to-trough decline	Silver MDD = -72% (Apr 2011 to Dec 2015)	Rolling/Period
Drawdown Duration	Time from peak to recovery	2011 peak: still not recovered as of 2025	Days/Months
Sharpe Ratio	Excess return per unit of volatility	Silver avg Sharpe ~0.3 (highly volatile)	Annualised
Sortino Ratio	Excess return per unit of downside vol	Penalises only negative volatility; better for asymmetric returns	Annualised
Calmar Ratio	Annualised return / max drawdown	Captures catastrophic risk; <0.5 is poor	Annualised
Skewness	Asymmetry of return distribution	Silver returns: slightly negative skew (crash risk)	Rolling
Kurtosis	Tail thickness of return distribution	Silver returns: kurtosis >5 (leptokurtic; fat tails)	Rolling
Beta to Gold	Sensitivity to gold price changes	Silver beta to gold typically 1.2–1.8 (higher beta)	Rolling 60D

Correlation Breakdown Risk	Risk of safe-haven failure	March 2020: silver-gold correlation went to ~0 during liquidation	Stress Period
Liquidity Risk	Volume/OI deterioration	Measured by bid-ask spread, volume decline, OI contraction	Real-time

Python Implementation - Comprehensive Risk Dashboard:

```
import pandas as pd
import numpy as np
from scipy.stats import skew, kurtosis, norm

data = pd.read_csv('silver_daily_ohlcv_2000_2025.csv',
parse_dates=['Date'])
data.set_index('Date', inplace=True)
returns = data['Returns_Pct'] / 100

# Rolling risk metrics (252-day window)
window = 252
data['Rolling_Vol'] = returns.rolling(window).std() * np.sqrt(252)
data['Rolling_Sharpe'] = (returns.rolling(window).mean() * 252) /
(returns.rolling(window).std() * np.sqrt(252))

# Sortino Ratio (downside deviation only)
downside = returns.copy()
downside[downside > 0] = 0
data['Rolling_Sortino'] = (returns.rolling(window).mean() * 252) /
(downside.rolling(window).std() * np.sqrt(252))

# Rolling Skewness and Kurtosis
data['Rolling_Skew'] = returns.rolling(window).apply(skew)
data['Rolling_Kurt'] = returns.rolling(window).apply(kurtosis)

# Maximum Drawdown (expanding window)
cumulative = (1 + returns).cumprod()
rolling_max = cumulative.expanding().max()
data['Drawdown'] = cumulative / rolling_max - 1
data['Max_Drawdown'] = data['Drawdown'].expanding().min()

# Historical VaR and CVaR
def historical_var(returns, confidence=0.95):
    return np.percentile(returns, (1 - confidence) * 100)

def historical_cvar(returns, confidence=0.95):
```

```
    var = historical_var(returns, confidence)
    return returns[returns <= var].mean()

var_95 = historical_var(returns.dropna(), 0.95)
cvar_95 = historical_cvar(returns.dropna(), 0.95)
print(f'Historical 95% VaR (daily): {var_95:.4f}
({var_95*100:.2f}%)')
print(f'Historical 95% CVaR (daily): {cvar_95:.4f}
({cvar_95*100:.2f}%)')
print(f'Maximum Drawdown: {data["Drawdown"].min():.4f}
({data["Drawdown"].min()*100:.2f}%)')
```

A7.3 Regime Detection Models

Silver markets exhibit distinct regimes: bull trends, bear trends, range-bound consolidation, and crisis/extreme volatility. Identifying the current regime is critical for selecting the appropriate forecasting model (LSTM performs well in trending markets, ARIMA in range-bound, ensemble with defensive weighting in crises).

Two approaches for regime detection:

Method 1 - Volatility-Based Classification:

```
# Simple regime classification using rolling volatility
vol_20d = returns.rolling(20).std() * np.sqrt(252)
vol_median = vol_20d.median()
vol_75 = vol_20d.quantile(0.75)
vol_90 = vol_20d.quantile(0.90)

data['Regime'] = 'Normal'
data.loc[vol_20d > vol_75, 'Regime'] = 'High Volatility'
data.loc[vol_20d > vol_90, 'Regime'] = 'Crisis'
data.loc[vol_20d < vol_20d.quantile(0.25), 'Regime'] = 'Low
Volatility'
```

Method 2 - Hidden Markov Model (HMM):

```
from hmmlearn import hmm

# Fit 3-state HMM to returns
model_hmm = hmm.GaussianHMM(n_components=3, covariance_type='full',
n_iter=100)
model_hmm.fit(returns.dropna().values.reshape(-1, 1))

# Predict states
states = model_hmm.predict(returns.dropna().values.reshape(-1, 1))

# Identify regimes by mean return and volatility of each state
for i in range(3):
    mask = states == i
    state_returns = returns.dropna().values[mask]
    print(f'State {i}: Mean={state_returns.mean()*252:.2f}% ann,
Vol={state_returns.std()*np.sqrt(252):.2f}% ann,
Days={mask.sum()}')
```

SECTION A8: MODEL EVALUATION FRAMEWORK

A8.1 Performance Metrics

Every model must be evaluated using a consistent set of metrics. The following table defines each metric and provides interpretation guidelines specific to silver price forecasting:

Metric	Formula	Unit	Interpretation for Silver
RMSE (Root Mean Square Error)	$\sqrt{(\text{mean}((y-\hat{y})^2))}$	USD per ounce	RMSE < $1.00 = Excellent; < $2.00 = Good; > $3.00 = Poor
MAE (Mean Absolute Error)	$\text{mean}(\lvert y-\hat{y}\rvert)$	USD per ounce	Less sensitive to outliers than RMSE; preferred when large errors are not disproportionately bad
MAPE (Mean Abs Pct Error)	$\text{mean}(\lvert y-\hat{y}\rvert/y) * 100$	Percentage	MAPE < 3% = Excellent; < 5% = Good; > 10% = Poor
Directional Accuracy	% of correctly predicted up/down	Percentage	> 55% = Useful for trading; > 60% = Excellent; < 50% = Worse than random
R^2 (Coefficient of Determination)	1 - SS_res/SS_tot	Unitless [0,1]	> 0.9 = Strong fit; caution: high R^2 on levels doesn't mean good returns forecast
Diebold-Mariano Test	Statistical test for forecast comparison	p-value	$p < 0.05$ = statistically significant difference between two models
Information Ratio	Active return / tracking error	Ratio	> 0.5 = Good forecaster; measures consistency of prediction quality

A8.2 Walk-Forward Validation Protocol

CRITICAL: Standard k-fold cross-validation is INVALID for time series data because it creates temporal data leakage (training on future data to predict the past). All models must use walk-forward (expanding window or rolling window) validation.

The validation protocol for this project:

Period	Date Range	Size	Purpose
Training Period	Jan 2000 – Dec 2017	4,696 trading days	Primary model fitting; hyperparameter selection
Validation Period	Jan 2018 – Dec 2020	753 trading days	Hyperparameter tuning; ensemble weight calculation; includes COVID stress test
Test Period	Jan 2021 – Dec 2025	1,257 trading days	Final evaluation only; NO model adjustments allowed based on test results
Walk-Forward Step	1 day (daily models)	Expand window by 1 day	Retrain periodically (every 60 days for LSTM; every day for ARIMA)

Walk-Forward Implementation:

```
def walk_forward_validate(model_fn, data, initial_train_size,
step=1, horizon=1):
    """
    Walk-forward validation with expanding window.
    model_fn: function that takes (train_data) and returns fitted
model
    Returns: predictions, actuals arrays
    """
    predictions, actuals = [], []
    for i in range(initial_train_size, len(data) - horizon, step):
        train = data[:i]
        test_val = data[i:i + horizon]
        model = model_fn(train)
        pred = model.forecast(horizon)
        predictions.append(pred[-1])
        actuals.append(test_val.iloc[-1])
    return np.array(predictions), np.array(actuals)
```

PART B: GAMIFIED SIMULATION BUSINESS CONCEPT

Note on the Simulation Concept: The gamified simulation concept presented in this section is provided as a reference framework. Students may choose to follow this concept or design their own original gamified simulation - with a different name, structure, narrative, or game mechanics - as long as it meets the assessment criteria outlined in Part F.

SECTION B1: COMMODITY TRADING ARENA - SIMULATION OVERVIEW

B1.1 Business Rationale And Problem Statement

Commodity Trading Arena is a browser-based gamified simulation platform designed to train and assess data analysts in the specialised discipline of commodity price analytics through realistic, time-pressured, scenario-driven challenges using Python/R machine learning models and Power BI dashboards. The player assumes the role of Chief Quantitative Analyst at a fictional global commodity trading firm - Argentum Capital Management - managing a $2 billion precious metals portfolio with silver as the primary holding. Each session simulates a full trading week in the life of a commodity data analyst: from receiving the Monday morning market brief to delivering weekly performance attribution and forecast updates to the Chief Investment Officer by Friday close.

The Training Gap This Platform Solves:

Traditional commodity analytics training relies on passive reading of broker research and market reports, occasional backtesting on static historical datasets without real-time pressure, and learning by making errors on live trading systems where losses are real. The consequences of errors in commodity price forecasting are severe: an incorrect forecast can lead to millions in mark-to-market losses; a misidentified trend can result in failed hedging programs; a missed regime change (e.g., the 2020 COVID crash or the 2021 Silver Squeeze) can wipe out months of accumulated returns.

India’s commodity derivatives market has grown significantly following SEBI’s regulatory reforms, with MCX silver futures among the most actively traded contracts globally. Global precious metals markets process hundreds of billions in annual turnover across physical and derivatives segments. Yet there is no standardised, scalable training platform for commodity data analysts that combines ML model building with real-time decision making under pressure.

Target Audience:

- Primary: Undergraduate and MBA students preparing for roles in commodity trading desks, hedge funds, and asset management firms
- Secondary: Working professionals in risk management, portfolio analytics, and commodity research seeking to upskill on ML-driven forecasting
- Tertiary: Commodity trading firms and banks seeking to assess quantitative analytics competency during hiring

B1.2 Simulation Architecture

Core Components:

- Market Replay Engine: Replays historical silver market data day-by-day with realistic intraday volatility, allowing players to experience the 2008 crash, 2011 bubble, COVID crash, and Silver Squeeze in real-time
- Forecast Challenge Engine: Presents the player with specific forecasting tasks - predict tomorrow's close, next week's direction, monthly volatility - scored against actual outcomes
- Portfolio Simulator: Virtual portfolio with realistic constraints (position limits, margin requirements, transaction costs) where players execute trades based on their model signals
- Model Building Workbench: Integrated Jupyter-like environment where players build, train, and deploy LSTM/ARIMA/XGBoost models with live scoring
- Dashboard Challenge: Time-pressured Power BI dashboard building exercises where players construct DAX measures under simulated CIO demands
- Crisis Mode: Randomised market shock events requiring immediate portfolio adjustment and model recalibration

Game Progression:

Level	Focus Area	Points Required	Portfolio Allocation
1 – Analyst Trainee	Basic technical analysis; SMA/EMA signals	1,000	Virtual demo account
2 – Junior Analyst	ARIMA model building; walk-forward backtesting	5,000	$10M allocation
3 – Analyst	LSTM development; multi-feature models	15,000	$50M allocation
4 – Senior Analyst	Ensemble methods; regime detection	30,000	$200M allocation
5 – Quant Strategist	Full pipeline; real-time forecasting system	50,000	$500M allocation
6 – Chief Quant	Crisis management; board-level reporting	100,000	$2B portfolio

SECTION B2: SCORING SYSTEM AND GAME MECHANICS

B2.1 Forecast Accuracy Scoring

Players earn points based on the accuracy of their price forecasts across multiple time horizons:

Challenge	Type	Scoring	Frequency
Next-Day Direction	Binary (up/down)	Correct = +50 pts; incorrect = -20 pts	Daily
Next-Day Price (MAPE)	Continuous	MAPE <1% = 200 pts; <2% = 150; <5% = 100; >5% = 25	Daily
Weekly Return Forecast	Continuous	Within 0.5% = 300 pts; within 1% = 200; within 3% = 100	Weekly
Monthly Volatility	Continuous	Within 2% of realised vol = 400 pts	Monthly
Regime Detection	Categorical	Correctly identify bull/bear/range = 500 pts	Monthly
Crisis Response	Time-limited	Portfolio protected within 2 hours = 1000 pts	Event-driven

B2.2 Portfolio Performance Scoring

Beyond forecast accuracy, players are scored on portfolio management metrics that mirror real-world performance evaluation:

- Total Return: Absolute P&L of the virtual portfolio; benchmark is buy-and-hold silver
- Sharpe Ratio: Risk-adjusted returns; >1.0 earns bonus multiplier; >2.0 earns 3x multiplier
- Maximum Drawdown: Penalty for drawdowns >10%; progressive deductions above 15%, 20%, 25%
- Win Rate: Percentage of profitable trades; >55% earns bonus points
- Risk Management: Proper stop-loss usage, position sizing, and hedge execution earn consistency bonuses

B2.3 Model Building Scoring

Category	Requirements	Max Points
Data Preprocessing	Stationarity testing, proper scaling, train/test split	150
Feature Engineering	Technical indicators, macro features, sentiment integration	200
Model Architecture	Appropriate model selection; correct hyperparameter tuning	250
Validation Method	Walk-forward validation; no data leakage; expanding window	200
Ensemble Construction	Multiple model combination with proper weighting	200
Code Quality	Clean, documented, reproducible code in Jupyter notebooks	150
Dashboard (DAX)	Interactive Power BI dashboard with 10+ advanced DAX measures	250
Documentation	Clear methodology write-up with performance analysis	100

SECTION B3: SCENARIO ENGINE DESIGN

The Commodity Trading Arena includes 12 pre-built market scenarios drawn from actual silver market events. Each scenario is parametrised to create realistic market dynamics while testing specific analytical competencies:

#	Scenario	Market Dynamics	Analytics Challenge	Difficulty
1	The Hunt Brothers Squeeze	Cornering the market; delivery squeeze	Position limit analysis; manipulation detection	Extreme
2	2008 Financial Crisis	Liquidity crisis; forced selling; margin calls	Stress testing; correlation breakdown; tail risk	Severe
3	2011 Silver Bubble	Parabolic rally to $49; crash to $30 in 5 days	Bubble detection; trend reversal; volatility regime	Severe
4	COVID Flash Crash	March 2020: $18 to $12 in 2 weeks	Crisis response; model breakdown; rapid recalibration	Severe
5	Silver Squeeze (Reddit)	Social media-driven demand; retail vs institutional	Sentiment analysis; alternative data integration	Moderate
6	Fed Rate Hike Cycle	2022–2023 aggressive tightening	Macro sensitivity; real rate modelling	Moderate
7	Indian Wedding Season	Seasonal demand surge (Oct–Feb)	Seasonality modelling; Prophet decomposition	Low
8	Solar Panel Demand Shock	Policy-driven green energy acceleration	Structural demand shift; supply inelasticity	Moderate
9	COMEX Delivery Squeeze	Physical demand exceeds warehouse stocks	Futures basis analysis; term structure	High
10	Dollar Crash Scenario	DXY falls 15% in 3 months	Currency impact; macro regime change	High
11	Mining Strike	Major producer country mine shutdown	Supply shock modelling; price spike duration	Moderate
12	Geopolitical Escalation	Major conflict; safe-haven flows	Correlation with gold; VIX spike response	High

PART C: CASE STUDIES & REAL-WORLD APPLICATIONS

SECTION C1: THE HUNT BROTHERS SILVER MANIPULATION (1979–1980)

C1.1 Background And Market Context

The Hunt Brothers silver manipulation remains the most dramatic episode in silver market history and is essential case study material for any commodity data analyst. Nelson Bunker Hunt and William Herbert Hunt, sons of Texas oil billionaire H.L. Hunt began accumulating silver in the mid-1970s as a hedge against inflation and potential dollar devaluation. By late 1979, the Hunts controlled an estimated 100–200 million troy ounces of silver - roughly one-third of the world's total deliverable supply - through a combination of physical holdings and futures contracts on COMEX and CBOT.

The price of silver rose from $6 per ounce in early 1979 to $49.45 on January 18, 1980 - a 724% increase in less than 12 months. The COMEX responded by implementing "Liquidation Only" trading rules (Silver Rule 7) on January 7, 1980, which prohibited new long positions and forced existing longs to either sell or take delivery. This triggered the infamous "Silver Thursday" crash on March 27, 1980, when the price collapsed from $21.62 to $10.80 in a single day, wiping out the Hunts' estimated $1.7 billion fortune.

C1.2 Data Analytics Lessons

- Concentration Risk Detection: CFTC position data showed extreme concentration before regulators acted; modern COT analysis can identify similar patterns early
- Open Interest Analysis: Total COMEX open interest expanded dramatically as the Hunts built positions; OI divergence from price is a manipulation signal
- Basis Distortion: The futures-spot basis collapsed into severe backwardation as physical demand (delivery notices) exceeded supply; basis analysis is critical for squeeze detection
- Volatility Regime Change: Implied and realised volatility spiked to unprecedented levels; GARCH models must account for such regime shifts
- Model Failure: Any ARIMA or moving average model trained on pre-manipulation data would have completely failed during the squeeze; this demonstrates the importance of regime-aware models and human oversight

C1.3 Analytical Exercise For Interns

Using the provided silver_sentiment_weekly.csv and silver_futures_contracts.csv datasets, interns must:

1. Build a concentration index using CFTC Non-Commercial Net Long as a percentage of Total Open Interest
2. Identify historical periods where the concentration index exceeded 2 standard deviations from its 3-year mean
3. Analyse the subsequent 30-day, 60-day, and 90-day silver price returns following extreme concentration readings
4. Build a DAX dashboard that visualises the COT Index alongside silver price with colour-coded extreme zones
5. Write a Python function that implements a real-time alert when positioning approaches extreme levels

SECTION C2: THE 2011 SILVER BUBBLE AND CRASH

C2.1 Background

In the aftermath of the 2008 Global Financial Crisis, unprecedented monetary stimulus (quantitative easing) by the Federal Reserve drove precious metals prices sharply higher. Silver rose from $8.88 in October 2008 to $48.70 in April 2011 - a 448% increase in 30 months. The rally was fuelled by inflation fears, a weak US dollar, surging investment demand (SLV ETF holdings exceeded 360 million ounces), and speculative momentum from retail investors.

The bubble burst spectacularly: silver fell from $48.70 to $33.55 in just 5 trading days (April 28 – May 5, 2011), a 31% crash. The CME Group raised margin requirements five times in quick succession, forcing leveraged speculators to liquidate. By December 2011, silver had fallen below $28, and by late 2015 it was trading near $13.70 - a 72% decline from the peak.

C2.2 Data Analytics Lessons

- Parabolic Price Detection: Log-periodic power law models and acceleration factors can detect unsustainable price trajectories
- Margin Impact Modelling: CME margin changes have an asymmetric impact - increases crush leveraged longs; decreases have muted effect
- Volume/Price Divergence: Volume peaked weeks before the price peak, signalling distribution; LSTM models should include volume features
- Gold/Silver Ratio Mean Reversion: The ratio fell to 32 (extreme) before reverting to 55+ - this ratio is a powerful mean-reversion signal
- ETF Flow Analysis: SLV outflows accelerated during the crash; ETF flow data is both a leading and lagging indicator

C2.3 Analytical Exercise

1. Implement a bubble detection algorithm using the Log-Periodic Power Law (LPPL) model on 2010–2011 silver data
2. Backtest a Gold/Silver ratio mean-reversion strategy: go long silver when ratio > 80, short when ratio < 40
3. Build a margin-impact model: correlate CME margin announcement dates with subsequent 5-day silver returns
4. Create a Power BI dashboard showing the 2011 bubble with RSI, Bollinger Band width, and volume divergence indicators

SECTION C3: COVID-19 SILVER CRASH AND RECOVERY (2020)

C3.1 Background

The COVID-19 pandemic triggered the most violent short-term moves in silver's modern history. Silver entered 2020 trading near $18, crashed to $11.77 on March 18 (a 35% decline in 3 weeks) as liquidity dried up and margin calls forced universal asset liquidation, then rallied to $29.26 by August 7 - a 148% recovery in less than 5 months. This V-shaped pattern tested every forecasting model's ability to handle unprecedented regime changes.

The crash phase was unusual because silver declined simultaneously with equities and gold (the "sell everything" liquidity crisis), breaking the traditional safe-haven correlation. The recovery phase was driven by unprecedented fiscal/monetary stimulus ($5+ trillion in US spending alone), inflation expectations, supply chain disruptions to mine production, and surging retail investment demand.

C3.2 Data Analytics Lessons

- Correlation Breakdown: The silver-gold correlation dropped to near zero during the March crash; models assuming stable correlations failed catastrophically
- Volatility Regime: Intraday ranges exceeded 10% on multiple days; standard deviation-based models vastly underestimated risk; heavy-tailed distributions (Student-t, GEV) are required
- Model Recalibration Speed: LSTM models trained on pre-COVID data required 2–4 weeks of new data to recalibrate; ensemble methods that included regime detection recovered faster
- Physical vs Paper Market Divergence: Physical silver premiums spiked to 50–100% above spot price as refineries shut down; the retail_bullion_premium field in sentiment data captures this dislocation
- Stimulus Response: M2 money supply growth accelerated from 6% to 25% YoY within 4 months; models incorporating M2 growth captured the recovery signal earliest

C3.3 Analytical Exercise

1. Using the 2020 data period, compare LSTM, ARIMA, Prophet, and XGBoost forecasting accuracy during the crash (Feb–March) and recovery (April–August) phases separately
2. Build a rolling correlation model between silver and gold; identify breakpoints where correlation fell below 0.3
3. Implement a stress test scenario in Power BI: "What if another COVID-like event causes a 35% crash?" showing portfolio impact
4. Create a physical market premium tracker using sentiment data; analyse lead-lag relationship between premiums and spot price
5. Build a regime detection model (Hidden Markov Model or simple volatility-based) that switches between crisis and normal forecasting modes

SECTION C4: REDDIT SILVER SQUEEZE (JANUARY–FEBRUARY 2021)

C4.1 Background

Following the GameStop short squeeze orchestrated by Reddit's r/WallStreetBets community in January 2021, attention pivoted to silver with the hashtag #SilverSqueeze trending globally. Reddit users argued that bullion banks held massive short positions on COMEX that could be forced to cover through coordinated buying of physical silver and SLV ETF shares. Silver spiked from $25 to $30 intraday on February 1 before falling back to $26 by February 5.

While the silver squeeze was far less successful than the GameStop squeeze (silver's market is vastly larger and more liquid), the episode demonstrated the growing influence of social media-driven retail investment flows on commodity markets and the importance of incorporating alternative sentiment data into forecasting models.

C4.2 Data Analytics Lessons

- Social Media as a Leading Indicator: Google Trends and social media mention counts spiked 48–72 hours before the price move; the sentiment_weekly data shows this pattern
- ETF Flow Mechanics: SLV received $1.5 billion in inflows in a single day; understanding ETF creation/redemption mechanics is critical for forecasting physical demand
- Market Microstructure: The COMEX raised margins and LBMA reported normal vault stocks, dampening the squeeze; institutional infrastructure analysis matters
- Mean Reversion: The spike was fully reversed within 4 trading days; extreme short-term deviations from fundamental value mean-revert quickly in liquid markets
- Feature Importance Shift: During normal markets, macro features dominate; during social media-driven events, sentiment features become primary drivers

PART D: 15-DAY PROJECT METHODOLOGY & DELIVERABLES

SECTION D1: OVERVIEW

The 15-day project is structured as a progressive deep-dive into commodity price forecasting using machine learning and advanced analytics. The intern will move from data exploration through feature engineering, model building, ensemble construction, dashboard development, and case study analysis to produce a comprehensive submission package. Each phase builds on the previous. Deliverables are cumulative and must be compiled into a final submission document.

The methodology uses AI-accelerated research and development - interns are encouraged to use Claude.ai, ChatGPT, Cursor, and similar tools for research synthesis, initial code generation, and concept validation. However, all models must be validated through proper backtesting with walk-forward methodology, all performance metrics must be computed on true out-of-sample data (no data leakage), and all analytical conclusions must be supported by statistical evidence. Unverified AI output treated as final will be penalised.

SECTION D2: DAY-BY-DAY BREAKDOWN

Day	Focus Area	Key Deliverable
1	Data Exploration & Domain Immersion	EDA report with statistical summary; stationarity test results; data quality assessment
2	Data Preprocessing Pipeline	Clean datasets; preprocessing functions; train/val/test splits; scaling pipeline
3	Feature Engineering (Technical)	20+ technical indicator calculations; feature correlation matrix; feature selection report
4	Feature Engineering (Macro + Sentiment)	Merged multi-source dataset; lagged features; macro regime classification
5	ARIMA/SARIMA Model	Auto ARIMA parameter search; walk-forward backtest; performance metrics
6	Facebook Prophet Model	Prophet with regressors; seasonal decomposition; changepoint analysis
7	LSTM Model Architecture	Multivariate LSTM with 60-day lookback; training curves; hyperparameter log
8	LSTM Model Tuning & Validation	Optimised LSTM; walk-forward validation; comparison vs ARIMA/Prophet
9	XGBoost & Random Forest	Tree-based models; feature importance rankings; SHAP analysis
10	Ensemble Model Construction	Weighted ensemble; stacking meta-learner; dynamic switching logic
11	Power BI Dashboard (Part 1)	Data model; star schema; 15+ core DAX measures; price analytics dashboard
12	Power BI Dashboard (Part 2)	Forecast comparison dashboard; risk metrics dashboard; macro correlation views
13	Case Studies & Documentation	Two detailed case study analyses; methodology write-up; performance comparison report
14	Validation & Polish	Final model validation; code cleanup; documentation review; presentation draft
15	Final Compilation & Handover	Final PDF report + Jupyter notebooks + Power BI file + data files + GitHub transfer

Day 1 - Data Exploration & Domain Immersion:

- Load all five CSV files into Jupyter Notebook; verify row counts match documentation (6,783 + 312 + 27,390 + 26 + 1,043)
- Generate statistical summaries: mean, median, std, min, max, skewness, kurtosis for all numeric columns
- Plot silver price from 2000–2025; identify visually the major regime changes (2008 crash, 2011 bubble, 2020 COVID, etc.)
- Run ADF and KPSS stationarity tests on raw prices and log returns
- Compute and plot ACF/PACF for log returns (40 lags) and squared returns (40 lags)
- Create correlation heatmap between silver daily features and macroeconomic indicators
- Check for missing values, outliers (>4 standard deviations), and data quality issues
- Research: Read at least 3 articles on silver market dynamics, COMEX trading mechanics, and India's MCX silver market

Day 2 - Data Preprocessing Pipeline:

- Build a modular preprocessing pipeline: load_data() → clean_data() → merge_sources() → create_features() → scale_data() → split_data()
- Handle missing values: forward-fill for macro data (monthly to daily); interpolate for isolated gaps; document all imputation decisions
- Implement proper train/validation/test split: 70% train (2000–2017), 15% validation (2018–2020), 15% test (2021–2025)
- Apply MinMaxScaler and RobustScaler separately; compare impact on LSTM convergence
- Create sequence generator function for LSTM: configurable lookback (30, 60, 90 days) and forecast horizon (1, 5, 20 days)
- Build data pipeline for XGBoost: tabular features with proper lag structure to prevent data leakage
- Save all preprocessed data to pickle files for reproducibility

Day 3 - Feature Engineering (Technical):

- Implement all 12 technical indicators from Section A4.1 (SMA, EMA, MACD, RSI, Bollinger, ATR, Stochastic, ADX, OBV, VWAP, Ichimoku, Fibonacci)
- Create lagged return features: 1, 2, 3, 5, 10, 20, 60 day lags
- Build volatility features: rolling 5/10/20/60-day standard deviation of returns; Parkinson volatility estimator using high-low range
- Compute feature correlation matrix; identify and handle multicollinearity (drop features with correlation > 0.95)
- Apply feature selection: mutual information, Boruta algorithm, or LASSO regularisation path
- Document each feature's calculation, source, and hypothesised relationship with silver price

Day 4 - Feature Engineering (Macro + Sentiment):

- Merge silver_macroeconomic_monthly.csv with daily data using date-based forward-fill; ensure no look-ahead bias (monthly data available only at month-end)
- Create macro regime classification: (1) Expansionary (PMI>50, rising M2), (2) Contractionary (PMI<50, falling employment), (3) Crisis (VIX>30)
- Engineer real interest rate feature: Fed_Funds_Rate minus CPI_YoY; compute 3-month and 12-month rate of change
- Merge silver_sentiment_weekly.csv using weekly forward-fill; create COT Index = (Current_Net_Long - 3yr_Min) / (3yr_Max - 3yr_Min)
- Build Google Trends momentum: week-over-week change and 4-week moving average of trends index
- Compute Gold/Silver ratio features: current ratio, 200-day SMA of ratio, z-score of ratio vs 3-year window
- Create silver price in INR per kg feature for MCX context: Silver_Price_USD * USD_INR * 32.1507
- Generate comprehensive feature importance ranking using mutual information and Random Forest; document top 30 features

Day 5 - Arima/Sarima Model:

- Run auto_arima with stepwise=True on log returns to find optimal (p,d,q) order; report AIC, BIC, HQIC
- Implement manual ARIMA grid search: p in [0,5], d in [0,2], q in [0,5]; visualise AIC landscape
- Perform residual diagnostics: Ljung-Box test for residual autocorrelation, Jarque-Bera for normality, ARCH-LM for heteroscedasticity
- If ARCH effects detected, extend to ARIMA-GARCH(1,1) model for volatility forecasting
- Implement walk-forward backtest: initial training window = 1000 days; step = 1 day; forecast horizon = 1 day
- Compute RMSE, MAE, MAPE, directional accuracy, maximum absolute error
- Plot actual vs predicted prices for the test period; create residual plots and QQ-plots
- Test SARIMA with weekly seasonality (s=5): compare AIC with non-seasonal ARIMA

Day 6 - Facebook Prophet Model:

- Prepare Prophet-formatted dataframe: ds (date), y (price), plus regressor columns
- Add external regressors: Gold_Price_USD, DXY_Index, VIX_Index, Real_Interest_Rate; forward-fill monthly to daily
- Configure Prophet: changepoint_prior_scale=0.15, yearly_seasonality=True, weekly_seasonality=True
- Run changepoint detection; visualise changepoint locations on price chart; correlate with known market events
- Generate seasonal decomposition plots: yearly pattern (wedding season, year-end), weekly pattern (Monday effect)
- Compare Prophet with and without external regressors: measure improvement in RMSE and MAPE
- Generate probabilistic forecasts with 80% and 95% confidence intervals; evaluate calibration
- Cross-validate using Prophet built-in: initial=3650 days, period=365 days, horizon=60 days

Day 7 - LTSM Model Architecture:

- Implement the complete LSTM pipeline from Section A3.2 with multivariate input (15 features, 60-day lookback)
- Test three architectures: (A) Single LSTM (128), (B) Stacked LSTM (128+64 with dropout), (C) Bidirectional LSTM (64)
- Train each architecture for up to 100 epochs with EarlyStopping (patience=10) and ReduceLROnPlateau
- Plot training and validation loss curves for each; identify overfitting patterns
- Experiment with lookback windows: 30, 60, 90, 120 days; record performance for each
- Implement learning rate finder: sweep from 1e-5 to 1e-1; plot loss vs learning rate; choose optimal
- Save the best model checkpoint (.h5 file) based on validation loss
- Visualise actual vs predicted with confidence bands on the validation set

Day 8 - LTSM Tuning & Validation:

- Hyperparameter optimisation: units (32,64,128,256), dropout (0.1,0.2,0.3), batch_size (16,32,64), learning_rate (0.0001,0.001,0.01)
- Implement walk-forward validation for LSTM: retrain every 60 days on expanding window; record computational cost
- Test multi-step forecasting: predict 1-day, 5-day, and 20-day ahead; compare RMSE across horizons
- Generate prediction error analysis: plot error distribution; identify conditions where LSTM fails (high volatility regimes)
- Compare LSTM across market regimes: bull markets, bear markets, range-bound, crisis periods
- Create comprehensive comparison: LSTM vs ARIMA vs Prophet across all metrics
- Document all hyperparameter experiments in a structured table with results

Day 9 - XGBoost & Random Forest:

- Implement XGBoost from Section A3.3 with walk-forward validation (TimeSeriesSplit, 5 folds)
- Implement Random Forest baseline: n_estimators=500, max_depth=10, min_samples_leaf=5
- Generate SHAP analysis for XGBoost: waterfall plot, beeswarm plot, dependence plots for top 5 features

- Analyse feature importance stability: run on 5 different time windows; check if top features are consistent
- Implement Bayesian hyperparameter optimisation using Optuna: 100 trials; objective = validation RMSE
- Test feature interactions: are there synergistic pairs (e.g., RSI + Volume, DXY + Gold)?
- Build interpretable decision tree (max_depth=4): visualise to understand key decision splits

Day 10 - Ensemble Model Construction:

- Implement Method 1: Simple average ensemble of ARIMA + LSTM + XGBoost + Prophet predictions
- Implement Method 2: Inverse-RMSE weighted ensemble using validation period RMSE as weights
- Implement Method 3: Stacking meta-learner - train Ridge regression on model predictions as features
- Implement Method 4 (bonus): Dynamic regime-switching - different weights for different volatility regimes
- Compare all ensemble methods vs individual models on held-out test set (2021–2025)
- Generate comprehensive performance summary: all 8 models (4 individual + 4 ensemble) across 5 metrics
- Implement simple trading strategy based on ensemble: long when predicted return > 0.5%, short when < -0.5%; calculate Sharpe
- Document final model selection rationale with statistical significance test (Diebold-Mariano)

Day 11 - Power BI Dashboard (Part 1):

- Import all 5 CSVs into Power BI; create star schema with date dimension table
- Build date dimension: Calendar, Year, Quarter, Month, Week, DayOfWeek, IsWeekend, TradingDayNumber
- Implement 15+ core DAX measures from Section A5: VWAP, Rolling Volatility, Max Drawdown, Sharpe, RSI, ATR
- Build Page 1 - Price Analytics: Candlestick-style chart, Volume bars, SMA overlay, RSI gauge
- Build Page 2 - Risk Analytics: Volatility heatmap by month/year, Drawdown waterfall, VaR calculation
- Apply colour scheme: primary blue (#1F4E79), silver (#C0C0C0), up-green (#228B22), down-red (#CC0000)
- Add date range slicer, year slicer, custom period selector (1M, 3M, 6M, 1Y, 5Y, All)

Day 12 - Power BI Dashboard (Part 2):

- Build Page 3 - Forecast Comparison: Actual vs Predicted for all models; model selector; error distribution
- Build Page 4 - Macro & Sentiment: Silver vs Gold/Silver ratio, Fed rate overlay, COT positioning, Google Trends
- Implement cross-dataset DAX: Silver-Gold Correlation, Supply-Demand Signal, COT Extremes Alert
- Add What-If Parameters: Fed rate impact scenario analysis using historical sensitivity
- Implement drill-through from monthly to daily; add bookmarks for key events (2008, 2011, 2020, 2021)
- Add dynamic titles; test all interactions and cross-filtering behaviour

Day 13 - Case Studies & Documentation:

- Select two case studies from Part C; write 1000+ word analysis each following the Background → Dynamics → Model Behaviour → Lessons template
- Run actual models on case study periods; show specific numbers where models succeeded and failed
- Create Power BI bookmarks for each case study; annotate key events on charts
- Begin final report: executive summary, methodology, data description, feature engineering approach
- Create architecture diagram: data sources → preprocessing → features → models → ensemble → dashboard

Day 14 - Validation & Polish:

- Final model validation: re-run all models on clean test set; verify no data leakage; cross-check metrics
- Code cleanup: remove dead code, add docstrings, ensure PEP 8 formatting, add type hints
- Create requirements.txt with exact package versions; write unit tests for core functions
- Complete final report: results, comparison tables, conclusions, limitations, future work
- Build 15-slide presentation deck for CIO-level audience
- Record 5–10 minute screen walkthrough video demonstrating dashboards and models

Day 15 - Final Compilation & Handover:

- Compile all deliverables into GitHub with folder structure: /data, /notebooks, /models, /dashboards, /reports
- Create comprehensive README.md: project overview, setup instructions, data dictionary, how to reproduce
- Export final report as PDF; save model weights (.h5, .json, .pkl); export Power BI file
- Create Excel performance summary with side-by-side model comparison
- Transfer GitHub repository ownership to @ZethetaIntern; submit checklist confirmation

SECTION D3: MANDATORY DELIVERABLES CHECKLIST

#	Deliverable	Description
1	EDA Report	Jupyter Notebook with statistical summaries, visualisations, stationarity tests
2	Preprocessing Pipeline	Modular Python code for data loading, cleaning, feature engineering, splitting
3	Feature Engineering Notebook	20+ technical indicators; macro features; correlation analysis; feature selection
4	ARIMA Model Notebook	Auto ARIMA; walk-forward backtest; residual diagnostics; RMSE/MAE/MAPE
5	Prophet Model Notebook	Prophet with regressors; seasonal decomposition; forecast visualisation
6	LSTM Model Notebook	Complete training pipeline; learning curves; hyperparameter search; evaluation
7	XGBoost Model Notebook	Feature importance; SHAP values; walk-forward validation; comparison table
8	Ensemble Model Notebook	Weighted average, stacking, dynamic switching; final ensemble performance
9	Power BI Dashboard (.pbix)	Data model with 5 tables; 20+ DAX measures; 4+ interactive report pages
10	Case Study Report	2 detailed case analyses (minimum 1000 words each) with data-driven insights
11	Final Report (PDF)	40+ page comprehensive report covering methodology, results, conclusions
12	Presentation Deck	15-slide PowerPoint summarising key findings for CIO-level audience
13	GitHub Repository	Private repo; clean commit history; requirements.txt; README with setup instructions

SUBMISSION FINAL CHECKLIST

#	Item	Requirements
1	GitHub Repository	PRIVATE; clean history; requirements.txt; README; all code files
2	Jupyter Notebooks (6+)	EDA, Preprocessing, ARIMA, Prophet, LSTM, XGBoost, Ensemble
3	Power BI File (.pbix)	Star schema; 20+ DAX measures; 4+ report pages; formatted
4	Data Files	All 5 CSVs in /data folder; preprocessed pickle files in /processed
5	Final Report (PDF)	40+ pages; all sections; professional formatting
6	Presentation (PPTX)	15 slides; CIO-level; findings and recommendations
7	Model Weights	Saved LSTM .h5; XGBoost .json; sklearn pickle files
8	Performance Summary	Excel sheet with all model metrics side-by-side
9	Case Study Reports	Two analyses; 1000+ words each; data-driven
10	Video Walkthrough	5–10 min screen recording demonstrating dashboards and models

PART E: NO-CODE TOOLS & PLATFORMS GUIDE

SECTION E1: DATA AND COMPUTATION TOOLS

Tool	Description	Use in This Project	Website
Google Colab	Free Jupyter environment with GPU	LSTM training; model prototyping	https://colab.research.google.com/
Kaggle Notebooks	Free compute with datasets	EDA; model building; competitions	https://www.kaggle.com/
Streamlit	Python web app framework	Deploy interactive forecast dashboards	https://streamlit.io/
Power BI Desktop	Microsoft BI tool (free)	DAX analytics; enterprise dashboards	https://www.microsoft.com/en-in/power-platform/products/power-bi/
DAX Studio	Free DAX query tool	DAX measure testing; performance tuning	https://daxstudio.org/
Tabular Editor	Free Analysis Services editor	Advanced DAX development; formatting	https://tabulareditor.com/
Excel (Microsoft 365)	Spreadsheet	FORECAST.ETS; Data Analysis Toolpak	https://www.office.com/
Orange Data Mining	Visual ML workflow builder	No-code model building and evaluation	https://orangedatamining.com/
KNIME Analytics	Open-source data pipeline	Visual data transformation; ML workflows	https://www.knime.com/
RapidMiner	Auto-ML platform	Automated model selection; deployment	https://altair.com/altair-rapidminer

SECTION E2: DATA SOURCES & RESEARCH PLATFORMS

Source	Data Available	Website
Silver Institute	Annual World Silver Survey; supply-demand data	https://silverinstitute.org/
COMEX/CME Group	Official futures data; margin requirements; delivery reports	https://www.cmegroup.com/
LBMA	London silver price; vault holdings; market reports	https://www.lbma.org.uk/
CFTC COT Reports	Weekly Commitments of Traders data; positioning	https://www.cftc.gov/MarketReports
FRED (Federal Reserve)	Economic data (Fed rate, CPI, M2, unemployment, DXY)	https://fred.stlouisfed.org/
World Bank	Commodity price forecasts; global economic indicators	https://www.worldbank.org/ext/en/home
MCX India	Indian silver futures; MCX spot prices	https://www.mcxindia.com/
RBI	Indian monetary policy; repo rate; forex data	https://rbi.org.in/
Yahoo Finance	Historical price data (via yfinance Python library)	https://finance.yahoo.com/
Quandl/Nasdaq Data Link	Premium commodity datasets; alternative data	https://data.nasdaq.com/
Google Trends	Search interest data for silver, precious metals	https://trends.google.com/trends/
SLV ETF (iShares)	ETF holdings data; creation/redemption activity	https://www.blackrock.com/corporate/global-directory#ishares

SECTION E3: DESIGN AND DOCUMENTATION TOOLS

Tool	Use	Website
Canva	Presentation design; infographics	https://www.canva.in/
Figma	UI/UX design for gamified simulation mockups	https://www.figma.com/
Mermaid.js	Flowcharts; architecture diagrams in markdown	https://mermaid.js.org/
Draw.io	Free diagram tool for system architecture	https://app.diagrams.net/
Notion/Obsidian	Project documentation; knowledge management	https://www.notion.com/
LaTeX (Overleaf)	Professional report formatting; mathematical notation	https://www.overleaf.com/
GitHub Pages	Project documentation hosting	https://docs.github.com/en/pages

PART F: SUBMISSION PROTOCOL

Projects are assessed using a comprehensive 1000-point rubric covering seven dimensions: Problem Understanding, Solution Quality, Research & Analysis, Presentation & Clarity, Innovation & Creativity, Feasibility & Practicality and CV alignment. Assessment is conducted through a combination of manual review and AI-powered evaluation.

END OF PROJECT DOCUMENT